

Find First & Last Position of an Element in a Sorted Array

① Problem Statement

Given a sorted array and a target element, find :

- First occurrence of the target.
- Last occurrence of the target.

If the target is not present, return $[-1, -1]$.

Example :

Input : $arr = [5, 7, 7, 8, 8, 10]$
target = 8

Output : $[3, 4]$

② Idea

Since the array is sorted, we use Binary Search. Instead of stopping when we find the target, we continue searching :

- First Occurrence : Search towards Left.
- Last Occurrence : Search towards Right.

③ Helper Function

We write one helper function that works for both cases.

helper(arr, target, flag)

flag = true $\rightarrow$ Find First Occurrence
flag = false $\rightarrow$ Find Last Occurrence

④ Algorithm

(1) Initialize

```
start = 0
end = arr.length - 1
ans = -1
```

(2) While

```
start <= end
```

(3) Find

```
mid = start + (end - start) / 2
```

(4) Case 1 : Target Found

```
arr[mid] == target
```

```
ans = mid
```

Now,

- If finding first occurrence

```
end = mid - 1
```

(Move left to check if another occurrence exists.)

- Else (finding last occurrence)

```
start = mid + 1
```

(Move right to check for another occurrence.)

(5) Case 2 : Target is Greater

```
arr[mid] < target
```

```
start = mid + 1
```

(Move Right)

(6) Case 3 : Target is Smaller

```
arr[mid] > target
```

```
end = mid - 1
```

(Move Left)

(7) Return

```
return ans;
```

⑤ Code

Helper Function

```
public static int helper(int[] arr, int target, boolean flag) {
    int start = 0, end = arr.length - 1, ans = -1;
    while (start <= end) {
        int mid = start + (end - start) / 2;
        if (arr[mid] == target) {
            ans = mid;
            if (flag) { // Find first occurrence
                end = mid - 1;
            } else { // Find last occurrence
                start = mid + 1;
            }
        } else if (arr[mid] < target) {
            start = mid + 1;
        } else {
            end = mid - 1;
        }
    }
    return ans;
}
```

Main Function

```
public static int[] searchRange(int[] arr, int target) {
    int first = helper(arr, target, true);
    if (first == -1)
        return new int[] {-1, -1};
    int last = helper(arr, target, false);
    return new int[] {first, last};
}
```

⑥ Dry Run

$arr = [5, 7, 7, 8, 8, 10]$ target = 8

First Occurrence (flag = true)

- mid = 2 $\rightarrow 7 < 8$
Move Right
 - mid = 4 $\rightarrow 8$ found
ans = 4
Search Left
 - mid = 3 $\rightarrow 8$ found
ans = 3
Search Left
 - End
- First = 3

Last Occurrence (flag = false)

- mid = 2 $\rightarrow 7 < 8$
Move Right
 - mid = 4 $\rightarrow 8$ found
ans = 4
Search Right
 - mid = 5 $\rightarrow 10 > 8$
Move Left
 - End
- Last = 4

Final Answer : $[3, 4]$

⑦ Time Complexity

Operation	Complexity
First Binary Search	$O(\log n)$
Second Binary Search	$O(\log n)$
Total	$O(\log n)$

⑧ Space Complexity

$O(1)$

⑨ Key Points to Remember

- The array must be sorted.
- Use one helper function with a boolean flag.
- Whenever the target is found :
 - Save the index in ans.
 - Continue searching.
- For first occurrence, move left ($end = mid - 1$).
- For last occurrence, move right ($start = mid + 1$).
- If the first occurrence is -1, the element is absent, so return $[-1, -1]$.

⑩ Interview Tip

Why don't we return immediately after finding the target?
Because there may be duplicate occurrences. We save the current index and continue searching : - Left half for the first occurrence.
- Right half for the last occurrence.